

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

TEAM PROJECT ON STUDENT INNOVATION AND STARTUP SEED GRANT ALLOCATOR

P R O J E C T R E P O R T

Submitted by

SWETHA D S

25ITR153

SUBADHARSINI V A

25ITR143

JAVA ASSIGNMENT

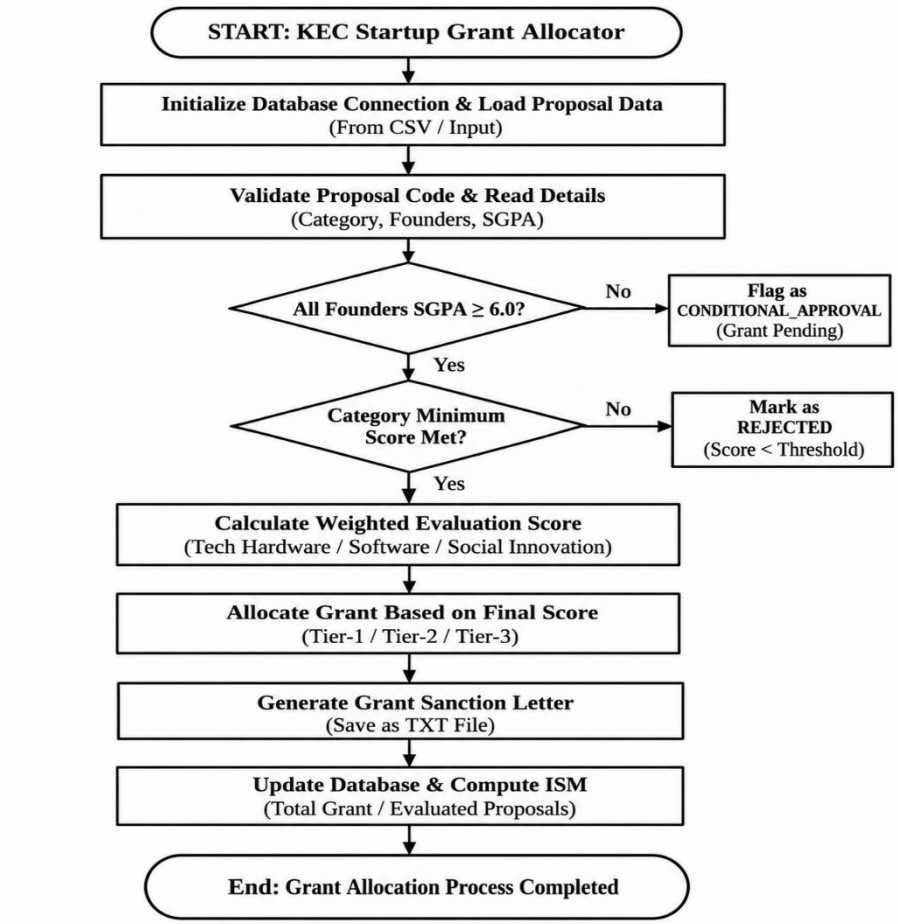
AIM:

To design and develop a KEC Student Innovation and Startup Seed Grant Allocator using Java and JDBC to evaluate student startup proposals, calculate weighted evaluation scores, verify academic eligibility based on SGPA, allocate seed grants according to predefined funding tiers, and generate grant sanction letters through a database-driven application.

OBJECTIVE:

The objective of this project is to develop a Java-based Student Innovation and Startup Seed Grant Allocator that evaluates startup proposals under different categories using weighted scoring criteria and student SGPA eligibility. It also aims to automate grant allocation, process proposal details using JDBC, CSV handling, and multithreading, and generate grant sanction letters through a cloud-hosted database.

FLOW CHART:



PROCEDURE :

- Create the required database and tables using the SQL schema file.
- Configure the database connection details in the Java application.
- Deploy the database on OCI VM 2.
- Configure the required VCN ingress rules and operating system firewall settings
- .Deploy the Java application on OCI VM 1.
- Establish remote database connectivity using JDBC.
- Register startup proposals and their founder details.
- Validate proposal codes using Java regular expressions
- Import bulk proposal information from CSV files.
- Record business plan evaluation scores during Phase 1.
- Record prototype technical audit scores during Phase 2.
- Record jury pitch scores and founder SGPA during Phase 3.
- Calculate the weighted score based on the startup category.
- Verify the minimum evaluation threshold.
- Check the academic eligibility of all founders.
- Assign the appropriate grant tier based on the final score.
- Process multiple evaluation sheets using multithreading.
- Generate grant sanction letters in TXT format.
- Store the final evaluation and allocation information in the database.
- Calculate the Incubator Success Metric based on the evaluation results.

DATABASE SCHEMA:

1.FOUNDERS TABLE:

```
mysql> SELECT * FROM founders;
+------------+------------+--------------+------+-----------------+
| founder_id | startup_id | student_name | sgpa | backlog_cleared |
+------------+------------+--------------+------+-----------------+
|          1 |          1 | Aarav Kumar  | 8.20 |               1 |
|          2 |          2 | Meena Ravi   | 7.80 |               1 |
|          3 |          2 | Arun S       | 5.80 |               0 |
|          4 |          3 | Kavya M      | 8.60 |               1 |
+------------+------------+--------------+------+-----------------+
4 rows in set (0.00 sec)
```

2.GRANT ALLOCATIONS TABLE:

```
mysql> SELECT * FROM grant_allocations;
+---------------+------------+--------------+------------+---------------------+---------------------+
| allocation_id | startup_id | grant_amount | grant_tier | approval_status     | disbursement_status |
+---------------+------------+--------------+------------+---------------------+---------------------+
|             1 |          1 |   1000000.00 | TIER-2     | APPROVED            | READY               |
|             2 |          2 |   1000000.00 | TIER-2     | CONDITIONAL_APPROVAL | ON_HOLD             |
|             3 |          3 |   1000000.00 | TIER-2     | APPROVED            | READY               |
+---------------+------------+--------------+------------+---------------------+---------------------+
3 rows in set (0.00 sec)
```

3.PITCH SCORES TABLE:

```
mysql> SELECT * FROM pitch_scores;
+----------+------------+-------------+----------------------+-------------------+---------------------+
| score_id | startup_id | final_score | category_threshold_met | eligibility_status | evaluated_at        |
+----------+------------+-------------+----------------------+-------------------+---------------------+
|        1 |          1 |       86.80 |                    1 | ELIGIBLE          | 2026-11-08 21:33:59 |
|        2 |          2 |       82.80 |                    1 | CONDITIONAL       | 2026-11-08 21:33:59 |
|        3 |          3 |       87.20 |                    1 | ELIGIBLE          | 2026-11-08 21:33:59 |
+----------+------------+-------------+----------------------+-------------------+---------------------+
3 rows in set (0.00 sec)
```

4.STARTUPS TABLES:

```
mysql> SELECT * FROM startups;
+------------+--------------+--------------------------+-----------------+--------------+---------------+------------+----
---------+-----------------+---------------------+
| startup_id | proposal_code | startup_name             | category        | business_score | prototype_score | pitch_score | imp
act_score | feasibility_score | created_at          |
+------------+--------------+--------------------------+-----------------+--------------+---------------+------------+----
---------+-----------------+---------------------+
|          1 | KEC-INC-1001  | Smart Irrigation Controller | TECH HARDWARE   |        82.00 |         88.00 |      91.00 |
    0.00 |            0.00 | 2026-11-08 21:25:37 |
|          2 | KEC-INC-1002  | Campus Skill Matcher       | SOFTWARE        |        78.00 |         85.00 |      88.00 |
    0.00 |            0.00 | 2026-11-08 21:25:37 |
|          3 | KEC-INC-1003  | Low Cost Water Testing     | SOCIAL INNOVATION |         0.00 |          0.00 |      86.00 |
   92.00 |           80.00 | 2026-11-08 21:25:37 |
+------------+--------------+--------------------------+-----------------+--------------+---------------+------------+----
---------+-----------------+---------------------+
3 rows in set (0.00 sec)
```

SOURCE CODE:

MAINAPP.JAVA:

```
package kec.allocator;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;
public class MainApp {
   private static final Scanner IN=new Scanner(System.in);
   private static final IncubatorDAO DAO=new IncubatorDAO();
   private static final FileProcessor FP=new FileProcessor();
   public static void main(String[] args) {
      System.out.println("KEC STUDENT INNOVATION & STARTUP GRANT ALLOCATOR");
      try(var c=DBConnection.getConnection()) {
         System.out.println("Database connected.");
      } catch(Exception e) {
         System.out.println("Database connection failed: "+e.getMessage());
         return;
      }
      while(true) {
         System.out.println("\n1. Import proposals from CSV\n2. Evaluate proposals\n3. View results\n4. Generate sanction letters\n5. Show ISM\n6. Exit");
         System.out.print("Choose: ");
         try {
            switch(IN.nextLine().trim()) {
               case "1" -> importCsv();
               case "2" -> evaluate();
               case "3" -> show();
```

```
                case "4" -> letters();
                case "5" -> System.out.printf(Locale.US,"ISM = Rs. %.2f%n",DAO.ism());
                case "6" -> {System.out.println("Goodbye.");return;}
                default -> System.out.println("Choose 1 to 6.");
              }
            } catch(Exception e) {
              System.out.println("Operation failed: "+e.getMessage());
            }
        }
    }
    private static void importCsv() throws Exception {
        System.out.print("CSV path [data/sample_proposals.csv]: ");
        String s=IN.nextLine().trim();
        Path p=s.isEmpty()?Path.of("data","sample_proposals.csv"):Path.of(s);
        List<StartupProposal> list=FP.readCsv(p);
        for(StartupProposal x:list) DAO.saveProposal(x);
        System.out.println("Imported "+list.size()+" proposal(s).");
    }
    private static void evaluate() throws Exception {
        List<StartupProposal> list=DAO.all();
        if(list.isEmpty()) {System.out.println("Import proposals first.");return;}
        ExecutorService
pool=Executors.newFixedThreadPool(Math.max(2,Runtime.getRuntime().availableProcessor
s()));
        try {
            List<Future<EvaluationResult>> fs=new ArrayList<>();
            for(StartupProposal p:list) fs.add(pool.submit(new ScoringWorker(p)));
            for(Future<EvaluationResult> f:fs) {
```

```
            EvaluationResult r=f.get();
            DAO.saveResult(r);
            System.out.printf(Locale.US,"%s: %.2f - %s%n",r.code(),r.score(),r.status());
          }
        } finally {
          pool.shutdown();
        }
        System.out.println("Parallel evaluation finished.");
    }
    private static void show() throws Exception {
        List<String[]> rows=DAO.results();
        if(rows.isEmpty()) {System.out.println("No results. Evaluate first.");return;}
        System.out.printf("%-15s %-25s %8s %-8s %13s %-22s %-
15s%n","CODE","STARTUP","SCORE","TIER","GRANT(Rs)","STATUS","DISBURSEM
ENT");
        for(String[] r:rows)
            System.out.printf(Locale.US,"%-15s %-25.25s %8s %-8s %13s %-22s %-
15s%n",r[0],r[1],r[2],r[3],r[4],r[5],r[6]);
    }
    private static void letters() throws Exception {
        List<String[]> rows=DAO.results();
        if(rows.isEmpty()) {System.out.println("Evaluate proposals first.");return;}
        FP.letters(rows,Path.of("output"));
        System.out.println("Letters saved in output/.");
    }
}
```

DB CONNECTION.JAVA:

```
package kec.allocator;

import java.io.*;

import java.nio.file.*;

import java.sql.*;

import java.util.*;

public final class DBConnection {

   private static final Properties P=load();

   private DBConnection() {}

   private static Properties load() {

      Properties p=new Properties();

      Path f=Path.of("config","db.properties");

      try {

         if(Files.exists(f)) {

            try(InputStream in=Files.newInputStream(f)) {

               p.load(in);

            }

         } else {


p.setProperty("db.url",System.getenv().getOrDefault("KEC_DB_URL","jdbc:mysql://localho
st:3306/kec_incubator?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
"));

            p.setProperty("db.user",System.getenv().getOrDefault("KEC_DB_USER","root"));


p.setProperty("db.password",System.getenv().getOrDefault("KEC_DB_PASSWORD",""));

         }

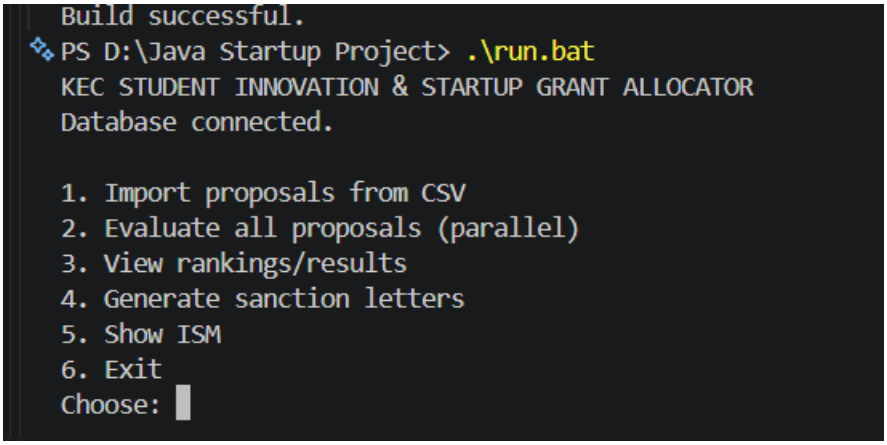
      } catch(IOException e) {

         throw new ExceptionInInitializerError(e);
```

```
        }
        return p;
    }
    public static Connection getConnection() throws Exception {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch(ClassNotFoundException e) {
            throw new IllegalStateException("MySQL Connector/J missing: place its JAR in
lib/",e);
        }
        return DriverManager.getConnection(
            P.getProperty("db.url"),
            P.getProperty("db.user"),
            P.getProperty("db.password","")
        );
    }
}
```

OUTPUT SCREENSHOTS:

MAIN MENU:



1.IMPORT PROPOSALS FROM CSV :

```
PS D:\Projects\KEC_Startup_Grant_Allocator>  .\run.bat
KEC STUDENT INNOVATION & STARTUP GRANT ALLOCATOR
Database connected.

1. Import proposals from CSV
2. Evaluate all proposals (parallel)
3. View rankings/results
4. Generate sanction letters
5. Show ISM
6. Exit
Choose: 1
CSV path [data/sample_proposals.csv]: sample_proposals.csv
Imported/updated 3 proposal(s).
```

2.EVALUATE ALL PROPOSALS (PARLLEL) :

```
1. Import proposals from CSV
2. Evaluate all proposals (parallel)
3. View rankings/results
4. Generate sanction letters
5. Show ISM
6. Exit
Choose: 2
KEC-INC-1001: 86.80 - APPROVED
KEC-INC-1002: 82.80 - CONDITIONAL_APPROVAL
KEC-INC-1003: 87.20 - APPROVED
Parallel evaluation finished.
```

3.VIEW RANKINGS/RESULTS :

```
1. Import proposals from CSV
2. Evaluate all proposals (parallel)
3. View rankings/results
4. Generate sanction letters
5. Show ISM
6. Exit
Choose: 3
CODE          STARTUP                   SCORE TIER        GRANT(Rs) STATUS               DISBURSEMENT
KEC-INC-1003  Low Cost Water Testing    87.20 TIER-2      1000000.00 APPROVED            READY
KEC-INC-1001  Smart Irrigation Controll 86.80 TIER-2      1000000.00 APPROVED            READY
KEC-INC-1002  Campus Skill Matcher      82.80 TIER-2      1000000.00 CONDITIONAL_APPROVAL ON_HOLD
```

4.GENERATE SNCTION LETTERS :

```
1. Import proposals from CSV
2. Evaluate all proposals (parallel)
3. View rankings/results
4. Generate sanction letters
5. Show ISM
6. Exit
Choose: 4
Letters saved in output/.
```

5.SHOW ISM :

```
 1. Import proposals from CSV
 2. Evaluate all proposals (parallel)
 3. View rankings/results
 4. Generate sanction letters
 5. Show ISM
 6. Exit
 Choose: 5
 ISM = Rs. 666666.67 per evaluated proposal
```

6.EXIT :

```
1. Import proposals from CSV
2. Evaluate all proposals (parallel)
3. View rankings/results
4. Generate sanction letters
5. Show ISM
6. Exit
Choose: 6
Goodbye.
```

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system accurately automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA)

REFERENCES:

1. Oracle, Java SE Documentation – JDBC Overview and Architecture. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ .

2. Oracle, Java SE API – HttpServer. https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html .

3. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/ .

4. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/8.0/en/ .

5. University Grants Commission, Guidelines for Choice Based Credit System (CBCS). https://www.ugc.gov.in/pdfnews/8023719_Guidelines-for-CBCS.pdf .

6. Chart.js, Chart.js Documentation. https://www.chartjs.org/docs/latest/ .

7. Tailwind CSS, Tailwind CSS Documentation. https://tailwindcss.com/docs .

8. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/ .

9. YouTube, Java JDBC with MySQL Tutorial Videos. https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial .

10. Anthropic, Claude AI Assistant (used for project ideas and guidance). https://claude.ai/ .

GITHUB:

LINK :	https://github.com/swetha181107/Java_Startup_Project